

Dynamic Profiles

Dynamic profiles allow a plugin to define profiles at runtime using JSON. Instead of creating profile files that don't change, it is possible to create profiles programmatically using a JSON structure.

If you are building a plugin that does not need to have dynamic profiles, you still have the choice of using the older method of static profile files, or send a JSON file that won't change at runtime when the plugin starts.

A plugin can only use either the static profile files, or the dynamic profiles, not both. Once a dynamic profile is sent to PG3/IoX, any static profile files will be ignored.

Dynamic profile requirements

To use Dynamic profiles, you need:

- IoX 6.0.6
- PG3x 3.4.5
- udi_interface 3.4.5

Using Dynamic Profiles

There are two methods that were added to the interface class to support dynamic profiles, `getJsonProfile()` and `updateJsonProfile()`:

`polyglot.getJsonProfile(options)`

Parameters:

- `options.waitResponse` **bool** (optional)
 - If `true`, the method waits for the response from PG3/IoX and returns the current profile.
 - If `false`, the method returns immediately with no data. To get the profile, the plugin must subscribe to the `polyglot.PROFILE` event.
 - **Default:** `false`

Example:

```
profile = polyglot.getJsonProfile({ "waitResponse": True })
```

Raises:

- `TimeoutError` – If `waitResponse` is `true` and no response is received from PG3/IoX.

Notes:

- `getJsonProfile()` returns all the nodedefs for the plugin, regardless of whether it was installed with static profile files or dynamic profiles. This can be useful for converting an existing static

profile into a dynamic profile.

- It is possible to retrieve the JSON profile format while still using static profile files.
- Please see Profile structure below for the JSON structure.

polyglot.updateJsonProfile(update, options)

Updates the JSON profile with additions, replacements, or deletions.

Parameters

update: dict A dictionary describing the updates to the profile.

Example update:

delete section rules:

- Optional. If omitted, nothing is deleted.
- For each category (`editors`, `nodedefs`, `linkdefs`):
 - "*" → Delete all items in that category.
 - List of IDs → Delete only the specified items.

Add/Replace Rules:

- Any `editors`, `nodedefs`, or `linkdefs` provided:
 - Replace existing items with the same `id`.
 - Add new items if no matching `id` exists.
 - Please refer to Profile structure below to see the structure of `editors`, `nodedefs` & `linkdefs`.

Raises:

- `ValueError` – If profile validation fails.
- `TimeoutError` – If `waitResponse` is `true` and no response is received from PG3/IoX.

options: dict (optional)

- `options.waitResponse` **bool** (optional)
 - If `true`, the method waits for the response from PG3/IoX.
This can be useful if you need to create a node based on this `nodedef`. The method returns when the profile has been created/updated.
 - If `false`, the method returns immediately with no data. If the plugin needs to know when the profile has been created, the plugin must subscribe to the `polyglot.PROFILE` event.
 - **Default:** `false`

Profile structure

Profile Object

Profile example:

Profile Fields

Field	Type	Description
<code>nodedefs</code>	<code>NodeDef[]</code>	Array of <code>NodeDef</code> . Nodes each has a reference to a <code>NodeDef</code> ID.
<code>editors</code>	<code>Editor[]</code>	Array of <code>Editor</code> . Properties and Command parameters have a reference to an editor.
<code>linkdefs</code>	<code>LinkDef[]</code>	Array of <code>LinkDef</code> . <code>NodeDefs</code> can have a reference to <code>LinkDefs</code> .

Notes:

- Please note that the JSON profile structure defined here is different from the JSON structure that the VS Code plugin uses.
- Currently, the VS Code plugin will generate profile files using the static profile format. It will eventually be updated to support this dynamic profiles structure.

NodeDef Object

Example:

NodeDef Fields

Field	Type	Description
<code>id</code>	<code>string</code>	<code>NodeDef</code> id.
<code>name</code>	<code>string</code>	<code>NodeDef</code> name
<code>properties</code>	<code>Property[]</code>	Properties for this node.
<code>cmds.sends</code>	<code>Cmd[]</code>	Commands the plugin can send.
<code>cmds.accepts</code>	<code>Cmd[]</code>	Commands the plugin can accept.
<code>linksctl</code>	<code>LinkDef[]</code>	Link definitions (controllers).
<code>linksrsp</code>	<code>LinkDef[]</code>	Link definitions (responders).
<code>desc</code>	<code>string</code> (Optional)	Description
<code>icon</code>	<code>string</code> (Optional)	UI icon identifier. Ref: Icons
<code>customicon</code>	<code>string</code> (Optional)	Future use.

Property Object

Defines a property of the node.

Property Fields

Field	Type	Description
<code>id</code>	<code>string</code>	Property ID (e.g., ST, GV0).
<code>name</code>	<code>string</code>	Property display name.
<code>editor</code>	<code>string</code>	Editor ID.
<code>desc</code>	<code>string</code> (Optional)	Description
<code>hide</code>	<code>bool</code> (Optional)	If <code>true</code> , property is hidden in UI.

cmd Object

Field	Type	Description
id	string	Command ID.
name	string	Command display name.
native	string	true/false indicating whether this is a command native to the device. Default: "true".
desc	string (Optional)	Description
format	string (optional)	This determines how this command is displayed in the program editor. Using format.
parameters	Parameter[] (optional)	Parameter definitions for the command.

Parameter Object

Field	Type	Description
id	string	Parameter ID. If empty string, this is the default parameter (and the only required parameter)
editor	string	This is the editor ID used for this parameter.
optional	bool (optional)	If true, this parameter is optional when sending the command.
init	string (optional)	Property Id. If specified, the UI showing the command will initialize it's default value from this property.
desc	string (Optional)	Description
name	string (optional)	There is rarely a name. Seems to be set when the parameter id is set to a non-empty value.

Editor Object

Editors define how values will be displayed or edited.

Example editor:

Editor Fields

Field	Type	Description
id	string	Editor ID used by node properties or commands.
ranges	Range[]	List of valid value ranges and constraints.

Range Object

A range object can be defined using min/max, or a subset

Using min/max:

Field	Type	Description
min	number	Lowest value allowed.

max	number	Highest value allowed.
prec	number (Optional)	Precision: Number of digits following the decimal point.
step	number (Optional)	Step increments the UI should use when increasing/decreasing the value.
uom	string	Unit of measure code.
names	Object (Optional)	Mapping of value to name. The object keys are the the values as string.
desc	string (Optional)	Description

Using subset:

Field	Type	Description
subset	string	Comma separated range-based allowed values. Example "0-5,8-10,20" Would allow values 0 to 5, 8 to 10 or 20.
uom	string	Unit of measure code.
names	Object (Optional)	Mapping of value to name. The object keys are the numbers as string, and the values are the names.
desc	string (Optional)	Description

LinkDef Object

Link definitions are used to describe the support for native links. They are scoped to a plugin and/or a product family supported directly by the eisy (e.g. Insteon, Z-Wave, etc.)

Field	Type	Description
id	string	LinkDef ID used by the NodeDef.
name	string	LinkDef name
protocol	string	The protocol supported (may contain any alphanumeric text or underline)
cmd	bool (Optional)	If true then any direct command for a responder can be used in the link (default is false). The linkdef must not specify any parameters.
format	string (optional)	This determines how this command is displayed in the scene when used as a responder. Using format.
parameters	Parameter[] (Optional)	This is used on Responders only. They represent the parameters that are passed when the scene is triggered. This is the same as parameters on commands, except for init which is not used here.
desc	string (Optional)	Description

All link definitions defined for a responder with the same protocol as supported by the controller are available for native links between a controller and a responder.

For example, there are two link protocols for Insteon, I_STD, and I_STD_ADV. I_STD is used for controllers that do not support retries, and I_STD_ADV is used for controllers that do. Therefore, any Insteon controller that supports the I_STD protocol can link to any responder that supports I_STD as well. The example below shows definitions for responders that are dimmers or thermostats.

Example linkdefs:

The following example shows how to define a link that can accept any command rather than just

parameters with an implied command. For example, the Aeon minimote allows native links where you can specify any command that the responder can accept. This makes any command supported in the "accepts" section of the nodedef for a responder available for use in the link (except for commands explicitly defined as non-native).

Formatting in Programs and Scenes

Each line of a program is formatted and displayed in different ways. Custom formatting using the "format" field are used for node conditions and commands. Scenes displaying commands in the responder section follow the same format.

Commands

The format for program command entries is like this:

```
param.id>/param text if omitted/param text if not omitted/ [... next parameter, ...]
```

Here's an example:

```
/level/${c}/to ${v}/ /ramprate// in ${v}/  
/
```

- The first character defines what character to use as the separator, normally '/' is used.

param.id

- Id of the parameter (e.g. 'level')
- This is blank for an unnamed parameter

param text if omitted

- String to show if the parameter was omitted

param text if not omitted

- String to show if the parameter was specified

The string for parameter text supports the following variables:

- `${c}` Name of the command
- `${v}` Formatted value of the parameter (including UOM)
- `${vo}` Formatted value of the parameter (without UOM)
- `${uom}` Formatted UOM without the value
- `${op}` Operator used (conditions only)

Command Formatting Examples

Assume commands are for node 'MyDevice'

1) A command with three named parameters, 'num', 'val', 'len' using this format:

```
/num//${c} Parameter ${v}/ /val/ default/ = ${v}/ /len// (${v} bytes)/
```

The following program action line would be shown for:

2) A command with one unnamed parameter using this format:

3) Another format example:

```
/level/${c}/to ${v}/ /ramprate// in ${v}/ /offtimer//, turn off ${v} later/
```

Migration from Static Profiles to Dynamic Profiles

If you have a plugin that uses the static profile format, the easiest way to migrate to the dynamic profile format is to retrieve your existing profile in the JSON format and update your plugin to start using this new format.

1. Retrieve the existing profile in the JSON format

Add this function to your code and call it before calling `polyglot.ready()`.

You may need to import the `time` module if you don't already have it.

Please note that the time delay is important. When the plugin starts, it will upload the profile files from the profile folder. The delay is to ensure that the profile files have been uploaded and have been processed by IoX.

Start your plugin, check the log, and copy-paste the profile somewhere temporary. It is recommended to use an online tool to pretty-print the JSON.

2. Update your plugin to use the JSON profile format

Use the profile retrieved in the previous step from your plugin log.

Load the profile in a variable

You can create a variable directly in your code to store the profile, or you can store it in a JSON file that your plugin will read.

Example, to load from a JSON file:

Send the profile to IoX

Create a function like this to send the profile to IoX, then call it on startup, before `polyglot.ready()`.

Stop using the static profile format

Remove any reference to `polyglot.updateProfile()` or `polyglot.checkProfile()`.

Also, rename or remove your profile folder. This step is **CRITICAL**.

If a profile folder exists, PG3x will send it to IoX on startup. If this process runs at the same time as a JSON profile update, you may get unexpected results.

3. Enhance your plugin to use fully dynamic profiles.

Once your plugin is updated to use the JSON profile format, you can use `polyglot.updateJsonProfile()` to add or remove individual nodedefs or editors.

You don't need to send the entire profile on each update. You can send only the changes.

You can, for example, call `polyglot.updateJsonProfile()` any time a new device is discovered. You can create the nodedef then the new node immediately after.